
Watch_SensorLogger

Release 3.3.0

Simon Brière

May 16, 2025

BINARY FILE FORMAT

1	Log file format	1
1.1	<i>session.oimi</i>	1
1.2	<i>watch_logs.txt</i>	1
2	Binary files format	3
2.1	Binary file header	3
2.2	Binary file data section	4
3	OpenTera data transfer	5
3.1	Configure data transfer	5
3.2	Device registration in OpenTera	5
3.3	Device association in OpenTera	5
4	Raw Accelerometer	7
4.1	Description	7
4.2	Settings	7
4.3	Binary file data format	7
5	Activity Detector	9
5.1	Description	9
5.2	Settings	9
5.3	Binary file data format	9
6	Battery Level	11
6.1	Description	11
6.2	Settings	11
6.3	Binary file data format	11
7	Beacons	13
7.1	Description	13
7.2	Settings	13
7.3	Binary file data format	13
7.4	A note about using Apple Watch devices as beacons	13
7.4.1	Eddystone	13
7.4.2	Apple Watch as a Beacon	14
8	Movement Disorders (Dyskinetic Symptoms)	17
8.1	Description	17
8.2	Settings	17
8.3	Binary file data format	17

8.4	Notes	17
9	GPS (Coordinates)	19
9.1	Description	19
9.2	Settings	19
9.3	Binary file data format	20
9.4	Notes	20
10	Raw Gyroscope	21
10.1	Description	21
10.2	Settings	21
10.3	Binary file data format	21
11	Headings (Processed magnetometer)	23
11.1	Description	23
11.2	Settings	23
11.3	Binary file data format	23
11.4	Notes	24
12	Health data	25
12.1	Description	25
12.2	Settings	25
12.3	Binary file data format	26
12.4	Notes	26
13	Raw Magnetometer	27
13.1	Description	27
13.2	Settings	27
13.3	Binary file data format	27
14	Processed Motion	29
14.1	Description	29
14.2	Settings	29
14.3	Binary file data format	29
15	Pedometer	31
15.1	Description	31
15.2	Settings	31
15.3	Binary file data format	31
15.4	Notes	32
16	Question Prompts	33
16.1	Description	33
16.2	Settings	33
16.2.1	Triggers	34
16.2.1.1	Events	34
16.2.1.2	Composite	34
16.2.1.3	Conditions	35
16.2.1.4	Example	35
16.3	Binary file data format	36
17	Sensoria Core	37
17.1	Description	37

17.2	Settings	38
17.3	Binary file data format	38
18	Movement Disorders (Tremors)	39
18.1	Description	39
18.2	Settings	39
18.3	Binary file data format	39
18.4	Notes	40
19	Indices and tables	41

LOG FILE FORMAT

Text files are used to store watch logs and session informations

1.1 *session.oimi*

This file contains a json structure with the following fields:

- *description*: A description structure of the recording
- *files*: A list of the files that are part of that recording
- *deviceId*: Device name that recorded the data
- *appVersion*: String representation of the SensorLogger app version when the recording was made
- *participant*: Participant name (if stored on watch) with the configuration used (separated with a >)
- *version*: File format version
- *logger_type*: A descriptor of the logger type used to create the dataset. Currently set to “Apple-Watch” for SensorLogger app.
- *timestamp*: String timestamp of the dataset recording (*format*: YYYY-MM-DD_hh:mm:ss)

1.2 *watch_logs.txt*

This file contains a log of events that occurred in the recording session. File is structured as a CSV file (tab separated) with the following columns:

- *Timestamp*: Timestamp of the event in UNIX format. The integer part of the timestamp is in seconds, while the decimal part corresponds to the milliseconds.
- *Log type*: ID of the log type, based on the following table:
 - 0: *General Error* - Error that happened in the recording session
 - 1: *General Information* - Information such as settings changed or connection states
 - 2: *General Warning* - Warning occurring in the recording session
 - 3: *Session Start* - Logging session start. One log (line) of that type will appear for each sensor that is being recorded.
 - 4: *Session Stop* - Logging session was stopped. One log (line) of that type will appear for each sensor that is being recorded.

- 5: *Device on charge* - Device was put on charge.
 - 6: *Device off charge* - Device was removed from charge.
 - 7: *Device low battery* - Device battery is under 5%.
 - 8: *Device storage low* - Device storage is under 100MB
 - 9: *Device storage full* - Device storage is full. Typically, this will also generate a lot of other errors in the log.
 - 10: *Device event* - Device event. Not used for now.
 - 11: *User event* - User entered event such as interaction with a button to mark data. Not used for now
- *Context*: Text context of what generated the log event. Typically, it will be the sensor name (see above), but could also be the function of the device that generated the event.
 - *Date & Time*: Textual representation of the timestamp, for readability purposes.
 - *Log*: Text describing the event. When the log is related to sensors, the log also contains the json settings of that sensor (see above) when started (log type = 3) and a summary of the recording session when stopped (log type = 4). This summary has the following fields:
 - *start_date*: Text representation of the time the recording started (no date)
 - *end_date*: Text representation of the time the recording was stopped (always None for now - to be fixed)
 - *logger_type*: *BinaryFile* or *CSVFile* depending on the format of the sensor's data files
 - *events_num*: Number of events (samples) that the sensor recorded in that session
 - *most_recent_event*: Unused for now.

BINARY FILES FORMAT

Sensor data are recorded into binary files.

Text file format for sensor data, while still available in the options, is not recommended and not directly supported in other tools such as [OpenIMU](#).

Binary data are formatted with a standard header and then a sensor specific data format

See *BinaryFormatter.swift* for the actual data serializer implementation.

2.1 Binary file header

- **File identifier** – 2 bytes : Always equal to 0xEAEA
- **File version** – 1 byte : Current version is 0x03
- **Participant ID**: UInt32 – 4 bytes: Optional participant ID. If not used, bytes should be set to 0x00000000
- **Sensor ID** – 1 byte : Id of the sensor data contained in that file. The current IDs are the following:
 - Battery: 1
 - Sensoria: 2
 - Heartrate: 3
 - Processed Motion: 4
 - Beacons: 6
 - GPS: 7
 - Raw accelerometers: 9
 - Raw gyrometers: 10
 - Pedometer: 11
 - Activity detector: 13
 - Question prompts: 14
 - HealthKit samples: 15
 - Headings: 16
 - Raw magnetometer: 17

- Tremors: 18
 - Dyskinetic symptoms: 19
- **Sensor settings descriptor size:** UInt32 – 4 bytes: Size of the json containing specific sensor settings
- **Sensor settings descriptor:** String (UTF8 encoding) – *Variable*: Sensor specific settings (such as sampling rate) in json format
- **Header end markers** – 2 bytes: Always equal to 0xEAEA

2.2 Binary file data section

Each binary file data format is defined in the specific page for each sensor:

- *Activity Detector*
- *Battery*
- *Beacons*
- *GPS*
- *Headings*
- *Health*
- *Heartrate*
- *Pedometer*
- *Processed Motion*
- *Raw Accelerometer*
- *Raw Gyroscope*
- *Raw Magnetometer*
- *Sensoria*
- *Question Prompts*
- *Tremors*
- *Dyskinetic symptoms*

OPENTERA DATA TRANSFER

SensorLogger app can be configured to transfer data to OpenTera, using a [PiHub](#) setup (for now).

3.1 Configure data transfer

1. Make sure you have properly configured the PiHub (especially with the correct server address and device registration key)
2. In the “Settings” menu of the app, properly set the “Server” field with the IP address of the PiHub (**Not the OpenTera server**) and the appropriate port (default: 8118)
3. Make sure the watch wifi network is the same as the PiHub, so that the hub can be reached by the watch

3.2 Device registration in OpenTera

1. In the “Settings” menu of the app, find the “OpenTera” section
2. Tap the “Register” button to register the watch (device) in OpenTera.
3. When the register process is successful, you’ll have to log in into OpenTeraPlus and go enable the device in the settings.
4. At the same time, make sure the device is associated correctly to the required site(s) / project(s)

3.3 Device association in OpenTera

1. In OpenTeraPlus, select the participant you want to associate the device to.
2. In the “Devices” tab, double-click on the device to associate. **Make sure to deassociate the device from other participants (answer yes when required)**

RAW ACCELEROMETER

ID: 9

4.1 Description

This sensor records the raw acceleration (unprocessed by the device).

4.2 Settings

- *frequency*: Sampling rate (in Hz)

4.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Accelerometer x-data**: Float32 – 4 bytes: Accelerometer data for x-axis (g)
- **Accelerometer y-data**: Float32 – 4 bytes: Accelerometer data for y-axis (g)
- **Accelerometer z-data**: Float32 – 4 bytes: Accelerometer data for z-axis (g)

ACTIVITY DETECTOR

ID: 13

5.1 Description

This sensor is based on the [CMMotionActivity](#) detector, which allows to detect the current activity type with a level of confidence.

5.2 Settings

None

5.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Detected Activities:** UInt8 – 1 byte: Detected activities and confidence level:
- **Bits 0-1:** Confidence level:
 - 00 : low
 - 01 : medium
 - 10 : high
- **Bit 2:** Automotive activity detected
- **Bit 3:** Cycling activity detected
- **Bit 4:** Running activity detected
- **Bit 5:** Stationary activity detected
- **Bit 6:** Walking activity detected
- **Bit 7:** Unknown activity detected

BATTERY LEVEL

ID: 1

6.1 Description

This sensor records the current battery level (in percentage) of the device and the current state (such as charging or unplugged) of the battery.

6.2 Settings

- *check_interval*: Interval (in seconds) between each battery reading

6.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Battery level**: UInt8 – 1 byte: Battery level in percentage (between 0 and 100, 0 is invalid / unknown state)
- **Battery state**: UInt8 – 1 byte: Battery state
 - 0: Unknown
 - 1: Unplugged
 - 2: Charging
 - 3: Full

BEACONS

ID: 6

7.1 Description

Recordings of external low-energy bluetooth (BLE) beacons complying with the Eddystone protocol.

7.2 Settings

- *namespace*: Namespace of beacons to record
- *known_beacons*: Specific beacons list to record. If empty [], record all in the namespace
- *monitor_all*: Display all beacons, even if some are filtered

7.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
 - **Beacon name**: 4 Char – 4 bytes: Beacon name string. XXXX is for undefined (unnamed) beacons.
 - **RSSI**: Signed byte – 1 byte: RSSI (signal strength) between -128 and 127.
-

7.4 A note about using Apple Watch devices as beacons

7.4.1 Eddystone

For the time being, the only supported beacon type is ‘Eddystone’, which is a standardized beacon advertisement frame and identification developed by Google.

Each beacon advertises the following in every advertisement frame data, [see protocol definition for more info](#)

- EddystoneUUID : 1B
- TxPower at 0 meter : 1B

- Namespace : 10B
- InstanceID : 6B

7.4.2 Apple Watch as a Beacon

For security reasons, the Apple Watch is **very** restricted on what it can advertise via BLE, as usual way more than the iPhone, which at first glance would seem to be equivalent. There have been many deadends trying to figure out the best way to scan for nearby watches, if it is even at all possible. Even if it was, continually identifying other watches in proximity is most definitely against TOS.

1. Act as a peripheral with fingerprinting info (eg. pack id in advertisement data, similar to Eddystone)
 - watchOS API doesn't offer access to the CBPeripheralManager API necessary to advertise
 - **if this is ever possible, it would be the easiest way to implement advertising self as a beacon**
2. Act as an iBeacon
 - Apple's own beacon implementation offered through the CoreLocation framework
 - Apple watch is one of the only devices that cannot be an iBeacon (i.e. again can't act as peripheral)...
3. Use the NearbyInteraction framework
 - Newish devices include the U1 chip for more precise distance and direction information between devices
 - While this API is available in Apple Watch, to start a NearbyInteraction session both parties need to get the other's `discoveryToken`
 - There is currently no way to directly communicate between Apple watches
 - All of the `following options` proposed by the Apple documentation are not available

To perform this discovery and handshake, the app implements custom code using a network technology you choose. Common examples are Core Bluetooth, Multipeer Connectivity, Watch Connectivity, or a custom server deployment. Over the network, the apps share specific data that the system requires to begin an interaction session, such as the session's `discoveryToken`.

 - `Multipeer Connectivity` would be the 'best' way to communicate between devices running the same application, but again *watchOS is the only apple device that cannot access it*
 - Custom server *might* be an option in certain deployments (e.g. there is a rpi accessible), but not as ubiquitous
4. Listen to already advertised BLE packets from the watch
 - The Apple Watch already periodically broadcasts some packets used for apple's own services (NearbyInteraction and Continuity Service)
 - The packets themselves do not have any real fingerprinting information that would allow another device uniquely identify the watch.
 - The UUID and MAC are rotated periodically
 - Through a BLE connect, we can get the device type and os version, but not its ID

- Surprisingly, when two devices that are on the **same apple account** are in vicinity, then the bluetooth manager fills up the device name with the name the user gave for this device.
 - This ‘feature’ doesn’t seem to be documented, and it is unclear what are the limitations.
 - Probably fetches DNS info based on the rotated MAC, probably internet needed (or access to user’s iphone) which means that in some deployments where the watch is standalone, this wouldn’t be an option.

MOVEMENT DISORDERS (DYSKINETIC SYMPTOMS)

ID: 19

8.1 Description

This system records the [movement disorders information](#) - Dyskinetic Symptoms, provided by Apple.

8.2 Settings

None

8.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Start Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision on which the measurement started
- **End Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision on which the measurement ended
- **Percent Likely:** Float32 – 4 bytes: Percentage of likeliness that there was dyskinetic symptoms over that period, between 0 and 1

8.4 Notes

- Data is analyzed / collected over a 60 seconds window.
- As identified by Apple, data is not valid on person who has not been diagnosed with a movement disorder problem.
- As always, with processed data from Apple, source is scarce on the accuracy of those data. Use wisely!

GPS (COORDINATES)

ID: 7

9.1 Description

This sensor records the user's position as reported by the GPS positioning system (and device). Since Apple Watch doesn't provide direct access to the GPS chip, this sensor is limited by what the watch is reporting (see "Notes" below).

9.2 Settings

Note: Not all settings are present in the recorded file json, depending mostly on the **GPS mode**.

- *mode*: GPS recording mode:
- **Continuous**: Positions are recorded as available, GPS is never stopped, sampling rate around 1Hz in ideal conditions
- **Ponctual**: Position are requested at the specified **period** interval
- **PowerCycling**: Periodically opens the GPS for **max_active_interval** seconds or until the **accuracy_threshold** is sufficient, whatever occurs first
- *period*: Interval of time, in seconds, to query GPS for a position (**Ponctual** mode only)
- *max_active_interval*: Interval of time, in seconds, to wait for a position (**PowerCycling** mode only)
- *accuracy_threshold*: Position accuracy threshold, in meters, to consider a position valid and stop acquisition (**PowerCycling** mode only)
- *beacon_proximity_idle_time*: Time, in seconds, for which to stop the GPS when a valid beacon is detected in the vicinity
- *accuracy*: Required **accuracy** for the position
- *activity_type*: GPS **activity type**. *None* activity type should (if Apple will) provide unfiltered GPS positions.
- **Other / None**: 1
- **Automotive Navigation / Car**: 2
- **Fitness**: 3
- **Other Navigation / Other**: 4

9.3 Binary file data format

- **Timestamp:** UInt64 – 8 *bytes*: Timestamp (Unix format) with milliseconds precision
- **Latitude:** Float32 – 4 *bytes*: Latitude position
- **Longitude:** Float32 – 4 *bytes*: Longitude position
- **Position accuracy:** Float32 – 4 *bytes*: Accuracy (in meters) for lat/long
- **Altitude:** Float32 – 4 *bytes*: Altitude (in meters) from sea level
- **Altitude accuracy:** Float32 – 4 *bytes*: Accuracy (in meters) for altitude
- **Speed:** Float32 – 4 *bytes*: Speed (meters per second)
- **Course:** Float32 – 4 *bytes*: Degrees relative to true north

9.4 Notes

- GPS data is never raw - the Watch OS processes it somehow, and thus no guarantee can be made to position not sent to the app (as filtered by the OS as not relevant).
- GPS position can be a drain on the battery - correct mode should be set according to recording needs.

RAW GYROSCOPE

ID: 10

10.1 Description

This sensor records the raw angular velocity (unprocessed by the device) from the gyroscope.

Important: Currently, raw angular velocity is not available on Apple Watch devices. When recording on such devices, this file will contain the **processed** gyroscope data.

10.2 Settings

- *frequency*: Sampling rate (in Hz)

10.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Gyroscope x-data**: Float32 – 4 bytes: Gyroscope data for x-axis (deg/s)
- **Gyroscope y-data**: Float32 – 4 bytes: Gyroscope data for y-axis (deg/s)
- **Gyroscope z-data**: Float32 – 4 bytes: Gyroscope data for z-axis (deg/s)

HEADINGS (PROCESSED MAGNETOMETER)

ID: 16

11.1 Description

This sensor records the headings processed from raw magnetometer (on device with such sensor). This is completely computed by the device itself and dependent on its configuration.

11.2 Settings

None

11.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **True Heading:** Float32 – 4 bytes: Heading in degrees relative to true north (0 = North, 180 = South). Only valid if “**True Heading Accuracy**” is positive.
- **True Heading Accuracy:** Float32 – 4 bytes: Accuracy in degrees of “**True Heading**”, with negative values representing unreliable or uncalibrated sensor.
- **Magnetic Heading:** Float32 – 4 bytes: Heading in degrees relative to magnetic north.
- **Magnetometer x-data:** Float32 – 4 bytes: x-value of magnetometer (microTeslas)
- **Magnetometer y-data:** Float32 – 4 bytes: y-value of magnetometer (microTeslas)
- **Magnetometer z-data:** Float32 – 4 bytes: z-value of magnetometer (microTeslas)

11.4 Notes

- In order to minimize data size, only headings with a difference > 5 degrees from the previous measurement are recorded.

HEALTH DATA

ID: 15

12.1 Description

Health data is provided to the device through the HealthKit. Little information is provided on how the data is computed. Thus, all relevant variables have been made available to the logger, but not all have been validated.

12.2 Settings

- *healthkitTypes*: List of HealthKit types to record. *Supported types are can be found here:*
 - activeEnergyBurned (kCal)
 - heartRate (bpm). See also *Heartrate*
 - restingHeartRate (bpm)
 - walkingHeartRateAverage (bpm)
 - stepCount (count). See also *Pedometer*
 - oxygenSaturation (%)
 - vo2Max (ml/kg/min)
 - heartRateVariability (ms)
 - bodyTemperature (°C)
 - basalEnergyBurned (kCal)
 - respiratoryRate (bpm)
 - appleExerciseTime (seconds)
 - appleStandTime (seconds)
 - appleMoveTime (seconds)
 - appleWalkingSteadiness (%)
 - walkingSpeed (m/s)
 - walkingStepLength (meter)

12.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision of the samples reception
- **Start Timestamp:** UInt64 – 8 bytes: Sample start timestamp (Unix format) with milliseconds precision
- **End Timestamp:** UInt64 – 8 bytes: Sample end timestamp (Unix format) with milliseconds precision
- **Type index:** UInt16 – 2 byte: Index of the type of this sample in the list of types **from the given settings list** (see above)
- **Value:** Float64 – 8 bytes: Value of the sample. Units are defined by the type of the sample (see above).

12.4 Notes

- Health related variables have variable sampling rates. When known, they have been noted above.
- Use with cautious: data hasn't been validated, and might require some extra information (such as wearer age, weight and height) to be accurate.

RAW MAGNETOMETER

ID: 17

13.1 Description

This sensor records the raw magnetic field (unprocessed by the device) from the magnetometer.

Important: Currently, raw magnetometer values are not available on Apple Watch devices. When recording on such devices, this file will contain the **processed** magnetic field data.

13.2 Settings

- *frequency*: Sampling rate (in Hz)

13.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Magnetometer x-data**: Float32 – 4 bytes: Magnetometer data for x-axis (microTeslas)
- **Magnetometer y-data**: Float32 – 4 bytes: Magnetometer data for y-axis (microTeslas)
- **Magnetometer z-data**: Float32 – 4 bytes: Magnetometer data for z-axis (microTeslas)

PROCESSED MOTION

ID: 4

14.1 Description

This feature records all the provided processed motion data (accelerometer, gyroscope), gravity vector and orientation of the device. Data has been cleaned up by the device.

14.2 Settings

- *frequency*: Sampling rate (in Hz)

14.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Accelerometer x-data**: Float32 – 4 bytes: Accelerometer data for x-axis (g)
- **Accelerometer y-data**: Float32 – 4 bytes: Accelerometer data for y-axis (g)
- **Accelerometer z-data**: Float32 – 4 bytes: Accelerometer data for z-axis (g)
- **Gravity vector x-data**: Float32 – 4 bytes: Gravity vector data for x-axis (g)
- **Gravity vector y-data**: Float32 – 4 bytes: Gravity vector data for y-axis (g)
- **Gravity vector z-data**: Float32 – 4 bytes: Gravity vector data for z-axis (g)
- **Gyroscope x-data**: Float32 – 4 bytes: Gyroscope data for x-axis (deg/s)
- **Gyroscope y-data**: Float32 – 4 bytes: Gyroscope data for y-axis (deg/s)
- **Gyroscope z-data**: Float32 – 4 bytes: Gyroscope data for z-axis (deg/s)
- **Attitude quaternion w-data**: Float32 – 4 bytes: w (q0) component of attitude quaternion
- **Attitude quaternion x-data**: Float32 – 4 bytes: x component of attitude quaternion
- **Attitude quaternion y-data**: Float32 – 4 bytes: y component of attitude quaternion
- **Attitude quaternion z-data**: Float32 – 4 bytes: z component of attitude quaternion

PEDOMETER

ID: 11

15.1 Description

This sensor records the provided Apple step counts, distance estimation, cadence and floors count. Data is written as soon as it is available, and only includes the count from the start of the session (typically when the device was removed from charger) up to the current time.

15.2 Settings

None

15.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Number of steps:** UInt32 – 4 bytes: Current total step count from the start of the session
- **Distance:** Float32 – 4 bytes: Current total estimated distance, in meters, from the start of the session. -1.0 if the value is not available on device
- **Average Active Pace:** Float32 – 4 bytes: Current average pace, in m/s, since the start of the session. -1.0 if the value is not available on device
- **Current Pace:** Float32 – 4 bytes: Current estimated pace, in m/s. -1.0 if the value is not available on device
- **Current Cadence:** Float32 – 4 bytes: Current cadence in steps per second. -1.0 if the value is not available on device
- **Floors Ascended:** Int32 – 4 bytes: Total number of floor ascended since the start of the session. -1.0 if the value is not available on device
- **Floors Descended:** Int32 – 4 bytes: Total number of floor descended since the start of the session. -1.0 if the value is not available on device

15.4 Notes

- All of those data are computed by the device itself, and as such no accuracy can be precisely established
- Data might be related to the health app configuration. Keep that in mind when lending watches to multiple participants.

QUESTION PROMPTS

ID: 14

16.1 Description

This system allows to display prompts on the device for user entered information. Those prompts are customizable for their question, answers and display modes. They can also be programmed to trigger based on various conditions (see below).

16.2 Settings

- *notifyInterval*: Interval between notification, in format 'hh:mm:ss'. Note that no notifications will play if the prompt is already in view.
- *prompts*: List of prompt configurations:
 - *id*: Unique identifier of the prompt
 - *trigger*: Trigger of the prompt (see below).
 - *question*: Question to ask to the user, as an UTF-8 string
 - *answers*: Configuration of the possible answers to the question
 - * *style*: Style of the answer. Can be one of the following:
 - *selection*: User can select only one answer, presented as buttons
 - *checkboxes*: User can select multiple answers, presented as buttons that show selection
 - *picker*: User can select only one answer, presented as a picker
 - *stepper*: User can select only one answer, presented as a stepper
 - *slider*: User can select only one answer, presented as a slider
 - * *choices*: List of all possible answers, as UTF-8 strings
 - * *acceptLabel*: For styles that require confirmation of selection (such as checkboxes), text to show on the confirmation button. Defaults to "OK" if not specified.<

16.2.1 Triggers

Triggers are meant to be as composable as possible, and can be combined to create complex conditions.

16.2.1.1 Events

- *Nil*: Never fires.
- *OnStart*: Fires as soon as device is activated, most likely session's start.
- *Every*: Fires every 'interval' time
 - interval: Time interval in "hh:mm:ss" format
- *Datetime*: Fires at the specified date
 - date: Date in "YYYY-MM-DD hh:mm:ss" format
- *Schedule*: Fires at the specified time of the day, on the specified days of the week
 - weekdays: List of days of the week to fire. If empty, fires every day. 'Day' format is any of "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun".
 - time: Time of the day to fire, in "hh:mm:ss" format.
- *QuestionAnswered*: Fires when the specified question is answered
 - questionID: Prompt ID of the question to monitor
 - withAnswer: Optional list of answers that will trigger this event. If empty, fires for any answer of the prompt.
- *Healthkit*: Query HealthKit for the specified type of data, and fires when the condition is met.
 - type: Type of data to query. See [Health values](#) for the list of supported types.
 - sumValues: If true, sum samples from start of session to trigger time for condition check. If false, only use the last sample.
 - Condition is one of: Always, Equal:{val: Float}, Nearly:{val: Float}, Within:{min: Float, max: Float}, Over:{val: Float}, Under:{val: Float}
- *NearBeacon*: Fires when the specified beacon is detected
 - namespace: Namespace of the beacon to detect
 - instanceID: Name of the beacon to detect

16.2.1.2 Composite

Triggers which apply a condition on another subtriggers. They all take 'trigger' as a parameter, which is the subtrigger to apply the condition on.

- *Starting*: Activates the subtrigger at the specified date.
 - "at": Date in "YYYY-MM-DD hh:mm:ss" format
- *Ending*: De-activates the subtrigger at the specified date.
 - "at": Date in "YYYY-MM-DD hh:mm:ss" format

- *Backoff*: When the subtrigger fires, it will be deactivated, and reactivated after the specified delay.
 - “for”: Time in “hh:mm:ss” format
- *AfterInactivity*: When the subtrigger fires, will ignore every following fires until at least the specified delay has been elapsed since the last fire. (ie. Like backoff, but subtrigger fires reset the wait delay)
 - “for”: Time in “hh:mm:ss” format
- *CycleActive*: Cycles the subtrigger on and off at the specified intervals. Full period is “on” + “off”.
 - “on”: Time in “hh:mm:ss” format to keep the trigger active
 - “off”: Time in “hh:mm:ss” format to keep the trigger inactive
- *Inactive*: Fires when the subtrigger has not fired for the specified delay.
 - “for”: Time in “hh:mm:ss” format
- *Delayed*: Fires a specified delay after the subtrigger has fired.
 - “by”: Time in “hh:mm:ss” format
- *Once*: Fires only once, the first time the subtrigger fires, then deactivates it.

16.2.1.3 Conditions

Group multiple triggers together.

- *AnyOf*: Fires when any of the subtriggers fires.
 - “triggers”: List of triggers to monitor
- *AllOf*: Fires when all of the subtriggers fired at least once, and then on every subsequent fire of any subtrigger.
 - “triggers”: List of triggers to monitor

16.2.1.4 Example

Given we have 3 beacons to fence the participants home, and we want to trigger a prompt only once when the participant first leaves home during a session. In an effort to reduce false positives of beacons not being scanned for environmental reasons (ie. walls), we want no events for a full 5 minutes before the prompt is triggered. Also, to reduce battery usage, we want to only scan for beacons for 5 minutes every hour.

```
"trigger": {
  "Once": {
    "trigger": {
      "CycleActive": {
        "on": "00:05:00",
        "off": "00:55:00",
        "trigger": {
          "Inactive": {
            "for": "00:04:55",
            "trigger": {
```

(continues on next page)

(continued from previous page)

```

        "AnyOf": {
            "triggers": [
                { "NearBeacon": { "namespace": "...", "instanceID": "..." } },
                { "NearBeacon": { "namespace": "...", "instanceID": "..." } },
                { "NearBeacon": { "namespace": "...", "instanceID": "..." } }
            ]
        }
    }
}

```

16.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision of the prompt answered by the user
- **Prompt ID size:** UInt16 – 2 bytes: Size of prompt ID string (N)
- **Prompt ID:** String – N bytes: Prompt identifier
- **Results size:** UInt16 – 2 bytes: Size of the results array (RN)
- **Results:** UInt16 – RN*2 bytes: Selected results (index of the selected answer in the list of answers)
- **Shown timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision of the prompt first displaying to user.

SENSORIA CORE

ID: 2

17.1 Description

This features records the data coming from bluetooth (BLE) connected [Sensoria Core](#) device.

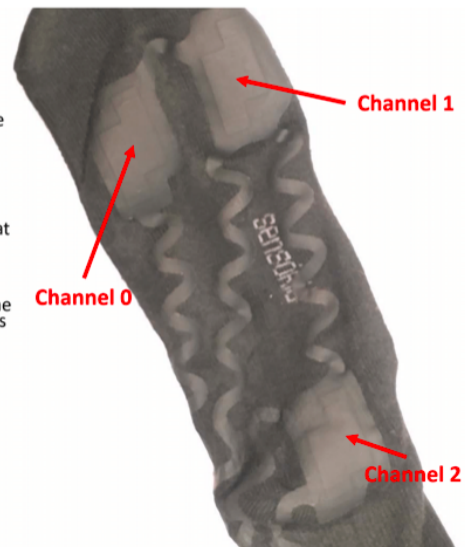
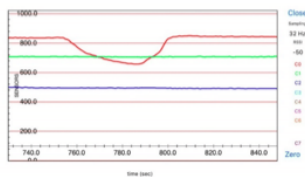
Sensoria Core / Core “Solo”: IMU

- Accelerometer:
 - 3-axis linear acceleration, measured in g's
 - Earth's gravity is reported in the measure
 - 10-bit resolution, Range: default +/-16g (available: +/-2g, +/-4g, +/-8g)
- Gyroscope:
 - 3-axis angular acceleration, measured in dps (degrees per second)
 - 10-bit resolution, Range: default +/-500 dps (available: +/-2000dps, +/-100dps, +/-245dps)
 - Gyroscope has a drift/offset $\neq 0$ at rest
- Magnetometer:
 - 3-axis magnetic north reference, measured in Gauss
 - 10-bit resolution, +/-16 G



Sensoria Core: Sock pressure sensors

- Channel Reference (right or left foot):
 - Channel 0: Metatarsus #5
 - Channel 1: Metatarsus #1
 - Channel 2: Calcaneus
- The sensor area is a square of 2x2 cm
- The reported value is a number from 0 to 1023 (Analog to Digital), indicating the voltage "captured" by the sensor
 - 1023: max resistance (sensor is disconnected due to interruption in the circuit)
 - 0: short circuit (sensor is bypassed by a short circuit)
 - A typical textile sensor acts within a range of 400-500 points
- The pressure is shown in the graph as a dip in the value compared to the value at rest
- The three sensors have different rest values
- The pressure indicator is relative rather than absolute; the more the pressure the less the value, but the value is not directly relatable to a precise psi value, unless calibration processes are introduced (available as custom service)



17.2 Settings

- *frequency*: Sampling rate (in Hz)
- *defaultDeviceSuffix*: Suffix of the Sensoria device that is scanned

17.3 Binary file data format

- **Timestamp**: UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Pressure channel #1**: Int16 – 2 bytes: Pressure sensor value for channel #1
- **Pressure channel #2**: Int16 – 2 bytes: Pressure sensor value for channel #2
- **Pressure channel #3**: Int16 – 2 bytes: Pressure sensor value for channel #3
- **Accelerometer x-data**: Float32 – 4 bytes: Accelerometer data for x-axis (g)
- **Accelerometer y-data**: Float32 – 4 bytes: Accelerometer data for y-axis (g)
- **Accelerometer z-data**: Float32 – 4 bytes: Accelerometer data for z-axis (g)
- **Gyroscope x-data**: Float32 – 4 bytes: Gyroscope data for x-axis (deg/s)
- **Gyroscope y-data**: Float32 – 4 bytes: Gyroscope data for y-axis (deg/s)
- **Gyroscope z-data**: Float32 – 4 bytes: Gyroscope data for z-axis (deg/s)
- **Magnetometer x-data**: Float32 – 4 bytes: Magnetometer data for x-axis (Ga)
- **Magnetometer y-data**: Float32 – 4 bytes: Magnetometer data for y-axis (Ga)
- **Magnetometer z-data**: Float32 – 4 bytes: Magnetometer data for z-axis (Ga)

MOVEMENT DISORDERS (TREMORS)

ID: 18

18.1 Description

This system records the [movement disorders information](#) - Tremors, provided by Apple.

18.2 Settings

None

18.3 Binary file data format

- **Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision
- **Start Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision on which the measurement started
- **End Timestamp:** UInt64 – 8 bytes: Timestamp (Unix format) with milliseconds precision on which the measurement ended
- **No Tremor Ratio:** Float32 – 4 bytes: Ratio of time where no tremor where detected, between 0 and 1
- **Slight Tremor Ratio:** Float32 – 4 bytes: Ratio of time where slight tremors where detected, between 0 and 1
- **Mild Tremor Ratio:** Float32 – 4 bytes: Ratio of time where mild tremors where detected, between 0 and 1
- **Moderate Tremor Ratio:** Float32 – 4 bytes: Ratio of time where moderate tremors where detected, between 0 and 1
- **Strong Tremor Ratio:** Float32 – 4 bytes: Ratio of time where strong tremors where detected, between 0 and 1

18.4 Notes

- Data is analyzed / collected over a 60 seconds window.
- As identified by Apple, data is not valid on person who has not been diagnosed with a movement disorder problem.
- As always, with processed data from Apple, source is scarce on the accuracy of those data. Use wisely!

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`